KANT-RAG · RETRIEVAL_A / RETRIEVAL_B

하이브리드 검색은 왜 몇 개만 가져올까

dense(뜻으로 찾기)와 sparse(단어로 찾기)를 합치는 하이브리드 검색이 단계마다 몇 개를 다루는지, 그리고 왜 그 개수로 줄이는지 정리했습니다. 숫자는 우리 프로젝트 설정 기준입니다.

먼저 답부터

일반적인 하이브리드 검색은 정해진 개수(후보)만 가져와서 합칩니다. 문서 전체에 점수를 매기는 방식은 흔하지 않습니다.

Qdrant, Elasticsearch, LangChain EnsembleRetriever 등 대부분의 도구가 dense 상위 N개와 sparse 상위 N개를 받아 합친 뒤, 그중 상위 k개를 씁니다. 우리 구성 중에는 **A · 기존**만 예외적으로 전체 청크에 점수를 매깁니다.

단계마다 줄어드는 깔때기

앞 단계는 싸고 빠른 방법으로 넓게 찾고, 뒤로 갈수록 비싸고 정확한 방법으로 좁힙니다. 아래는 advanced 컬렉션(청크 약 9만 개), hybrid + rerank 기준입니다.

① **후보 검색** — 약 90,000 → 각 100개

dense와 sparse가 각자 가장 가까운 N개를 찾습니다. 벡터 색인(ANN)은 전체와 일일이 비교하지 않고도 상위 N개를 밀리초 단위로 찾습니다. 하위권은 어차피 최종에 들어올 가능성이 거의 없어 버립니다.

② **점수 합치기** — 최대 200개 → 50개

두 목록을 모아 dense와 sparse 점수를 함께 봅니다(min-max 가중합 또는 RRF). 후보만 다루므로 계산이 거의 들지 않습니다. 리랭커 없이 hybrid만 쓰면 여기서 바로 상위 5개를 고릅니다.

③ **리랭커** — 50개 → 5개

리랭커(cross-encoder)는 질문과 청크를 한 쌍씩 통째로 읽어 판단합니다. 정확하지만 비쌉니다. 9만 개에 쓰면 질문 하나에 몇 시간이 걸려서, 앞 단계가 추린 50개에만 씁니다(약 0.5초).

④ **답변 LLM 입력** — k = 5개

입력이 길어지면 비용과 시간이 늘고, 관련 없는 내용이 섞이면 답이 흐려집니다. 발제 조건("k는 세 방식에서 같게")에 맞춰 모든 구성을 5개로 맞췄습니다.

우리 세 구성은 몇 개씩 다루나

구성	① 후보	② 합치는 방식	③ 리랭커 후보	④ 최종 k	방식
A · 기존	**전체 청크** (advanced 약 9만 개 전부에 점수)	전체 기준 min-max 가중합 (dense 0.7 : sparse 0.3)	50	5	전체 방식
A · Qdrant	dense 상위 100 + sparse(bge-m3) 상위 100	후보 안에서 min-max 가중합 (0.7 : 0.3)	50	5	후보 방식 (표준)
B	dense 상위 30 + BM25 상위 30	RRF (k = 60)	50	5	후보 방식 (표준)

세 구성 모두 마지막에는 같은 5개만 모델에 넘깁니다. 다른 점은 그 5개를 고르기 전에 몇 개를 놓고 비교하느냐입니다.

A · 기존은 왜 전체를 봤나

처음에는 Qdrant 컬렉션에 sparse 벡터가 없었습니다. sparse 점수를 프로그램이 직접 계산하려면 청크를 모두 받아 와야 했고, 받아 온 김에 전체에 점수를 매긴 것입니다. 이후 컬렉션에 sparse 벡터(m3, bm25)가 들어가면서 표준 방식인 A · Qdrant로 바꿀 수 있게 됐습니다.

전체 방식 (A · 기존)

후보를 놓칠 일이 없어 advanced에서 가장 안정적입니다(hybrid + rerank Hit@5 0.897). 대신 서버 시작이나 첫 사용 때 청크를 모두 받아 와야 해서 advanced에서 몇 분이 걸리고 메모리를 많이 씁니다. 문서가 늘수록 부담이 커집니다.

후보 방식 (A · Qdrant, B)

시작 대기가 거의 없고 가볍습니다. 문서가 늘어도 질문마다 후보 N개만 다룹니다. 대신 dense 후보에도 sparse 후보에도 못 든 문서는 아예 검토되지 않으므로 N이 중요한 설정값입니다.

개수가 너무 적거나 많으면

단계	너무 적으면	너무 많으면
① 후보 N	정답이 후보에 아예 못 들어옵니다. advanced에서 A · Qdrant가 A · 기존보다 조금 밀린 원인으로 보입니다.	거의 손해가 없습니다. Qdrant가 후보를 고르는 시간은 거의 그대로입니다.
③ 리랭커 후보 50	리랭커가 고를 정답이 없습니다.	리랭커 시간이 비례해서 늘어납니다 (100개면 약 1초).
④ 최종 k = 5	답에 필요한 근거가 빠집니다.	비용이 늘고 답이 흐려지며, 발제의 비교 조건(k 동일)이 깨집니다.

서비스(client)에서는 한 번 더 있습니다

사전 판단이 복합 질문을 하위 질문(최대 3개)으로 쪼개면, 검색 질문마다 위 과정을 한 번씩 거쳐 각각 상위 5개를 얻습니다. 결과를 순위대로 번갈아 합쳐 **최대 12개** (config/generation/rag.yaml의 max_docs)를 LLM에 넣습니다. 그래서 복합 질문은 단일 질문보다 응답이 2배쯤 오래 걸립니다(서버 기준 약 4~5초 → 8~14초). 평가 실험 (langfuse_experiment.py)은 질문을 쪼개지 않아 항상 5개입니다.

수치 출처: 2026-11-12 retrieval_a / retrieval_b 비교 평가(골든 데이터셋 192문항, 발제 기준 문서 단위 Hit@5). 설정 파일: config/retrieval_a/hybrid.yaml · rerank.yaml, config/generation/rag.yaml, retrieval_b Settings.